



中华人民共和国通信行业标准

XX/T s—XXXX

自动化软件测试能力成熟度模型

Maturity Model for Automated Software Testing Capability

[点击此处添加与国际标准一致性程度的标识]

(征求意见稿)

2024 年 11 月 1 日

XXXX - - XX 发布

XXXX - XX - 实施

中华人民共和国工业和信息化部 发布

目次

前 言III

自动化软件测试能力成熟度模型 1

1. 范围 1

2. 规范性引用文件 1

3. 术语和定义 1

 3.1 1

 3.2 1

 3.3 1

4. 缩略语 2

5. 自动化软件测试能力成熟度模型总体架构 2

6. 自动化软件测试能力成熟度模型能力分级规则 2

 6.1. 分级概述 2

 6.2. 分级规则 3

7. 测试对象（新增） 3

 7.1. 代码层 3

 7.2. 接口层 4

 7.3. UI 层 5

8. 测试类型（新增） 6

 8.1. 功能测试 6

 8.2. 性能测试 6

 8.3. 安全测试 7

 8.4. 兼容性测试 8

9. 测试自动化 9

 9.1. 资产自动化（新增） 9

 9.1.1. 测试用例自动化 9

 9.1.2. 测试脚本自动化 10

 9.1.3. 测试数据自动化 11

 9.2. 流程自动化（新增） 12

 9.2.1. 任务管理 12

 9.2.2. 过程监控 12

 9.3. 环境自动化 13

 9.4. 结果自动化（新增） 13

 9.5. 风险管理 14

 9.5.1. 缺陷管理 14

 9.5.2. 失败根因分析 15

 9.6. 测试效果 15

10. DevOps 一体化协同 16

 10.1. 敏捷管理 16

 10.2. API 管理 16

10.3. 评审管理..... 16

10.4. 版本管理..... 16

10.5. 代码管理..... 16

 10.5.1. 代码统一管理与版本控制16

 10.5.2. 代码质量管理17

 10.5.3. 代码安全管理17

10.6. 制品管理..... 19

10.7. 安全管理..... 19

10.8. 流水线管理..... 20

前 言

本文件按照GB/T 1.1—2020《》给出的规则起草。

请注意本文件的某些内容可能涉及专利。本文件的发布机构不承担识别这些专利的责任。

本文件由中国通信标准化协会提出并归口。

本文件起草单位：

本文件主要起草人：

自动化软件测试能力成熟度模型

1. 范围

本标准规定了自动化软件测试能力成熟度模型分级要求，覆盖测试对象、测试类型、测试自动化、DevOps 一体化协同四大核心方向自动化测试领域能力要求。

本标准适用于各行业企业对其组织或项目的测试能力成熟度进行评估，包括软件测试工具平台的提供方，以及软件测试体系建设方。针对不同使用场景，本文件对其技术指标的要求略有不同。

2. 规范性引用文件

下列文件对于本文件的应用是必不可少的。凡是注日期的引用文件，仅所注日期的版本适用于本文件。凡是不注日期的引用文件，其最新版本（包括所有的修改单）适用于本文件。

GB/T 38634.1-2020 系统与软件工程 软件测试 第1部分：概念和定义

3. 术语和定义

下列术语及定义适用于本文件

3.1

错误 error

系统运行中出现的可能导致系统崩溃或者暂停运行的非预期问题。

[来源：GB/T 39412-2020，3.1.6]

3.2

测试用例 test cases

前置条件、输入（包括操作步骤）和预期结果的集合，用于驱动测试项的执行以满足测试目标，测试目标包括正确实现、错误识别、检查质量和其他有价值的信息。

[来源：GB/T 38634.1-2020，3.48]

3.3

灾难备份 back for disaster recovery

为了灾难恢复而对数据、数据处理系统、网络系统、基础设施、技术支持能力和运行管理能力进行备份的过程。

4. 缩略语

下列缩略语适用于本文件：
API：应用编程接口（Application Programming Interface）
DSL：动态脚本语言（Dynamic Script Language）

5. 自动化软件测试能力成熟度模型总体架构

本文件所定义的自动化软件测试能力成熟度模型，如图1所示：



图 1 自动化软件测试能力成熟度模型

6. 自动化软件测试能力成熟度模型能力分级规则

6.1. 分级概述

自动化测试能力分级划分为基础级、增强级和先进级。各级别对应的具体能力要求见第7-11章。各级别服务能力描述如下：

- 基础级：能够基本实现功能要求；
- 增强级：在基础级能力上，具备较高级的功能实现；
- 先进级：在增强级能力上，具备行业内领先的功能实现。

6.2. 分级规则

基本分级单元为功能模块，每个模块都有 1 级、2 级、3 级的能力要求项。每级满足 90% 的能力项即可认为达到该级别要求，根据参评模块的通过级别分为基础级、增强级和先进级。

7. 测试对象（新增）

7.1. 代码层

1 级：

- 1) 应具备单元测试工具库，包含常用的单元测试框架（如 JUnit、JUnit、pytest 等）；
- 2) 应支持单元测试用例编写和执行，能够覆盖关键功能模块的核心逻辑；
- 3) 应支持代码格式化和基本的代码规范检查；
- 4) 宜支持自动化流水线执行单元测试，将单元测试集成到持续集成/持续部署（CI/CD）流程中，实现自动触发和执行。

2 级：

- 1) 应集成静态代码分析工具（如 SonarQube、ESLint、Pylint 等），在单元测试执行前检测代码中的潜在缺陷和规范问题；
- 2) 应支持测试用例与代码变更的自动关联（如基于 Git、SVN 等工具代码提交触发对应测试）；
- 3) 应支持多语言、多框架的单元测试，确保不同技术栈下的测试兼容性；
- 4) 应具备 Mock 或 Stub 工具（如 Mockito、EasyMock、WireMock 等），用于模拟外部依赖或复杂对象行为，确保单元测试的独立性和可重复性；
- 5) 应支持在容器化环境（如 Docker）或虚拟化环境中运行单元测试，确保测试环境的一致性和隔离性；
- 6) 应获取单元测试用例执行后的语句/分支覆盖率。

3 级：

- 1) 应支持自动化流水线执行单元测试，将单元测试集成到持续集成/持续部署（CI/CD）流程中，实现自动触发和执行；
- 2) 应支持单元测试案例的断言检查，避免出现无断言或无效断言的情况；
- 3) 应实现被测代码的覆盖率动态分析（如代码行覆盖、支覆盖率、条件覆盖率、语句覆盖），

可集成代码覆盖率工具（如 JaCoCo、Cobertura、Coverage.py 等）；

- 4) 重要系统的重要模块应达到 80%代码覆盖率；一般系统最低限度应不低于 30%代码覆盖率；
- 5) 应具备单元测试覆盖率阈值准入机制，当覆盖率达标时方可推进后续流程；
- 6) 应支持代码的版本控制和变更影响分析，能够评估代码变更对系统的影响；
- 7) 宜集成安全漏洞扫描工具，对代码进行安全检测，并提供修复建议。

7.2. 接口层

1 级：

- 1) 应支持单接口自动化测试与多接口串联流程测试；
- 2) 应支持多种接口协议，如 tcp、http、微服务框架等；
- 3) 应支持多种报文格式，如 json、xml、form 等；
- 4) 应支持对响应报文、响应头、响应码设置断言以及数据库验证等；
- 5) 应支持参数管理与使用，并支持参数的作用域设置，如全局变量、局部变量等；
- 6) 应支持逻辑控制，如循环、判断、时间间隔配置等；
- 7) 应支持数据驱动，且数据不应采用固定值直接写入代码的方式，而应预留灵活调整的接口或配置机制；
- 8) 应支持测试数据的动态生成与清理。

2 级：

- 1) 应支持复杂的接口测试场景，例如并发测试、数据驱动测试、负载测试；
- 2) 应支持脚本的复制、引用等复用功能，并支持引用后批量更新；
- 3) 应支持导入、录制等功能辅助脚本建设；
- 4) 宜集成安全测试工具（如 OWASP ZAP、Burp Suite），对接口进行安全扫描（如 SQL 注入、XSS、认证绕过等）。

3 级：

- 1) 应支持多环境切换执行；
- 2) 应支持接口性能的基准测试和对比分析：提供接口性能的基准测试功能，并能够对比不同版本或不同环境下的接口性能；
- 3) 应支持接口安全性的全面测试，例如于 SQL 注入、XSS 攻击、CSRF 攻击等安全漏洞的测试；
- 4) 宜支持接口测试覆盖率与业务场景的映射分析；
- 5) 宜支持当接口请求和响应结构发生变化时的自动识别。（如：新版接口应该能兼容老版的接口）

- 6) 具备流量回放测试能力，包括流量捕获与存储、精准流量回放、环境适配与模拟能力等；
- 7) 宜支持接口性能测试（如响应时间、吞吐量、并发用户数），并与性能测试工具（如 JMeter、Gatling 等）集成。

7.3. UI 层

1 级：

- 1) 应支持基本的 UI 元素（如输入框/下拉框/单选框等）定位和操作，能够模拟用户的基本交互例如：点击、输入、选择等；
- 2) 应支持多种主流浏览器（如 Chrome、Firefox、Microsoft Edge 等）。

2 级：

- 1) 应支持跨平台兼容性测试的基本配置：提供跨平台（如不同操作系统、不同分辨率、不同终端）兼容性测试的基本配置选项；
- 2) 应支持获取界面内容，并做为参数传递至后续步骤；
- 3) 应支持操作时间间隔设置、等待元素加载完成时间设置等；
- 4) 宜支持跨 iframe 或跨域窗口的元素操作；
- 5) 宜支持元素属性的模糊匹配（如部分匹配文本、正则表达式匹配），并允许设置重试阈值（如元素加载超时重试次数）；
- 6) 宜支持基于动态属性（如随时间变化的 ID、class 名）或上下文关系（如父子元素、兄弟元素）的元素定位；
- 7) 宜支持多种断言方式，包括 OCR 文本识别、图片比对等，同时支持验证元素的状态（如可见性、可点击性、选中状态等）；
- 8) 可支持模拟复杂用户行为，如拖拽、滑动、缩放、右键菜单、键盘快捷键组合等。

3 级：

- 1) 应支持多种元素定位方式，如属性、坐标、图片、文字识别等；
- 2) 应支持 UI 测试的录制与回放功能，提供 UI 测试的录制功能，并能够回放测试用例，以便快速定位问题；
- 3) 应支持将通用操作（如等待、检查界面等）封装为函数，以及将常用用例（如登录、查询等）制作成公共脚本，供测试人员直接引用；
- 4) 宜支持步骤级别的重试，提高页面自动化测试的稳定性；
- 5) 可支持基于自然语言描述进行的无脚本 UI 自动化；
- 6) 可支持基于 AI 驱动的视觉回归测试；

- 7) 可支持与用户体验监测工具（如 RUM）集成。

8. 测试类型（新增）

8.1. 功能测试

1 级：

- 1) 基于需求文档，实现核心功能点的自动化脚本覆盖，验证正向流程是否符合预期；
- 2) 应支持基础断言机制（如验证界面元素、API 响应状态码等）；
- 3) 应验证固定输入对应的固定输出（如输入正确密码登录成功）；
- 4) 应验证显式错误提示是否触发并符合需求；
- 5) 应支持在单一测试环境（如开发环境）中运行测试，支持基础的环境配置。

2 级：

- 1) 应支持数据驱动测试（参数化输入）、跨模块流程测试及部分异常场景覆盖；
- 2) 应支持跨功能模块串联测试及业务规则组合，验证完整业务流程的正确性；
- 3) 应基于需求反例自动生成边界值、无效输入等场景；
- 4) 应进行支持跨系统数据一致性检查；
- 5) 宜在功能测试中同步验证业务指标（如订单金额计算精度等）。

3 级：

- 1) 应支持复杂脚本化测试用例的执行，验证系统端到端功能是否符合需求；
- 2) 应支持跨平台（Web 端/移动端/后台等）、多协议（HTTP/消息队列等）的混合场景测试；
- 3) 应具备模拟第三方服务的挡板能力，进行第三方服务集成验证；
- 4) 应具备生网络配置能力，可根据配置进行弱网测试。

8.2. 性能测试

1 级：

- 1) 应根据场景选择不同的参数、配置方案。通过工具完成基础性能指标验证（如响应时间、吞吐量、并发用户数等）；
- 2) 应支持基础性能指标阈值配置（如响应时间/TPS/错误率）；
- 3) 应在测试环境中模拟真实的用户行为，覆盖关键业务流程的性能测试场景；

- 4) 可通过简单的脚本或工具实现性能测试的定时执行或手动触发。

2 级:

- 1) 应具备收集测试过程数据（如 TPS、响应时间、服务器资源使用率等），可视化展示测试过程；
- 2) 应支持支持阶梯加压、峰值负载、异常流量模拟（如网络抖动、服务中断）等测试模式；
- 3) 应支持对性能测试结果进行多维度分析，包括但不限于响应时间分布、资源利用率（CPU、内存、网络）、数据库性能瓶颈（如自动识别慢查询 SQL 并关联调用链）等指标；
- 4) 应支持性能基线管理能力，能够对比不同版本或阶段的性能测试结果，发现性能变化；
- 5) 应支持集成 CI/CD 工具实现自动化触发，支持测试任务并行执行；

3 级:

- 1) 应支持全链路性能压测，覆盖从测试数据打标、性能测试脚本编写、影子测试库表的环境准备、性能测试指标的实时监控，实现端到端的性能压测。
- 2) 应支持跨平台、跨设备、多协议（如 HTTP/WebSocket/MQTT）的性能测试能力，如支持 Web、移动端、API 等多种应用场景；
- 3) 应支持分布式压测与全链路追踪（如微服务调用链分析），结合工具定位跨系统性能瓶颈；
- 4) 宜实现高负载场景下的服务自愈能力验证（如自动扩缩容响应时间）；
- 5) 宜提供混沌工程集成能力（故障注入场景下的性能稳定性验证）；
- 6) 可支持性能预测能力，能够基于现有测试结果推测系统在更大用户量或更高负载下的表现。

8.3. 安全测试

1 级:

- 1) 应能完成 OWASP Top 10 基础漏洞验证（含 SQL 注入/XSS/CSRF 等自动化探测）；
- 2) 应集成静态代码安全扫描工具（如 SonarQube、Checkmarx）至 CI/CD 流水线，并具备代码提交安全检查功能，阻挡高危漏洞代码合并；
- 3) 应能验证核心业务权限矩阵正向/反向越权测试。
- 4) 应建立安全测试环境的基础隔离机制，物理隔绝测试数据与生产环境。；
- 5) 应能验证基础安全配置（如 HTTPS 加密/密码策略/会话超时）有效性；
- 6) 应输出漏洞风险等级分类清单（按 CVSS 评分分级展示）；
- 7) 测试完成后应至少包含两项产出物：安全测试报告、安全缺陷记录。

2 级:

- 1) 应集成 SAST+DAST 工具链并实现 CI/CD 流水线安全扫描卡点;
- 2) 应支持安全测试用例的动态参数化工具集成 (如 SQLMap、Burp Suite 等), 并对输入验证、权限绕过等场景的开展自动化模糊测试;
- 3) 应对安全测试结果实现与缺陷管理系统 (如 JIRA、Bugzilla、Tapd、禅道等) 的联动, 能够自动触发修复工单实行跟踪闭环;
- 4) 可支持安全测试覆盖范围, 包括 API 安全测试 (如 OAuth 授权漏洞等)、配置安全测试 (如默认密码、开放端口等) 及供应链安全测试 (如依赖库漏洞扫描等);
- 5) 可支持自动化渗透测试工具集成 (如 Metasploit、OWASP ZAP 等), 并生成攻击树分析报告。

3 级:

- 1) 应具备操作系统/数据库/中间件组件的安全配置基线检查能力;
- 2) 应实施容器镜像漏洞扫描与供应链攻击模拟 (如恶意依赖注入);
- 3) 宜验证漏洞修复闭环有效性 (含自动化回归测试集成);
- 4) 宜构建安全测试知识图谱, 基于历史漏洞数据自动生成测试策略 (如针对高风险模块的强化扫描规则);
- 5) 宜实现安全测试的混沌工程实践, 通过自动化故障注入 (如网络劫持、权限降级等) 验证系统容错能力;
- 6) 宜支持安全测试智能分析引擎, 利用 AI 模型对漏洞成因进行根因分析 (如代码片段溯源、攻击链还原等);
- 7) 宜支持安全测试左移机制, 在需求设计阶段通过自动化威胁建模 (如 STRIDE 模型) 预判风险, 并生成安全测试用例。

8.4. 兼容性测试

1 级:

- 1) 应实现主流浏览器 (如 Chrome、Firefox、Safari 等) 及操作系统 (如 Windows、macOS、Linux) 的自动化兼容性测试, 覆盖核心功能场景;
- 2) 应支持屏幕分辨率的自动化适配测试, 覆盖主流设备 (如 PC、移动端) 的常见分辨率 (如 1920×1080、1366×768 等);
- 3) 应支持移动端应用基础设备测试, 覆盖主流厂商 (如华为、小米、苹果等) 的硬件型号及操作系统版本 (如 Android 12+、iOS 15+等);

2 级:

- 1) 应支持辅助功能测试（如屏幕阅读器兼容性等）、网络环境模拟（如弱网、断网重连等）及多语言环境测试（如 RTL 布局、本地化字符显示）；
- 2) 应支持对不同设备特性（如摄像头权限、GPS 定位等）的自动化配置与验证；
- 4) 应支持兼容性测试结果的自动化报告自动生成，并清晰标注不兼容场景及附带失败用例的截图/日志；
- 3) 宜支持除主流操作系统、浏览器、终端等测试外，网络类型、特殊异形屏、高碎片化安卓设备特定测试规则等测试维度，并标明具体数字或参数。

3 级:

- 1) 应实现兼容性测试维度的具体范围应定期更新，建议根据生产统计数据或互联网调查机构统计数据、推荐以季度为周期进行更新；
- 2) 宜实现兼容性测试的混沌工程实践，通过自动化故障注入（如模拟旧版浏览器内核、异常系统权限）验证系统容错能力。

9. 测试自动化

9.1. 资产自动化（新增）

9.1.1. 测试用例自动化

1 级:

- 1) 应支持测试用例的增删改查；
- 2) 应支持用例分类（模块、优先级、测试类型等）；
- 3) 应具备用例导入导出功能，支持 Excel/CSV 等格式的批量导入导出；
- 4) 应支持用例与需求的关联；
- 5) 应支持基础版本控制（如历史版本回溯）。

2 级:

- 1) 应支持用例自动化生成，如基于需求文档、接口定义自动生成基础用例框架；
- 2) 应支持用例复用与批量操作；
- 3) 应支持用例执行历史记录与趋势分析；
- 4) 应支持自动化用例与手工用例的统一管理；
- 5) 应提供用例覆盖率统计功能，如需求覆盖率；

- 6) 应具备用例评审流程，支持多人在线批注和版本审核，宜支持用例评审状态跟踪。

3 级：

- 1) 应提供用例统计报表，展示用例数量、执行情况、缺陷分布等关键指标，辅助决策；
- 2) 应支持用例库的全局检索与知识图谱关联；
- 3) 应集成需求管理工具（如 Jira、禅道），实现用例与需求的双向追溯；
- 4) 应支持测试结果与缺陷管理系统（如 Bugzilla、Jira、禅道）的自动关联与闭环；
- 5) 应支持用例变更影响范围自动分析，如识别受用例变更影响的测试计划；
- 6) 宜支持自然语言交互形式创建用例，如通过 Chatbot 对话生成测试场景用例；
- 7) 宜支持用例智能推荐（基于代码变更或需求变更）。

9.1.2. 测试脚本自动化

1 级：

- 1) 应具备脚本创建与编辑能力，可手动编写或通过简单引导创建测试脚本，支持常见的脚本语法高亮显示与基础错误提示；
- 2) 应具备多维度脚本分类管理能力，支持按项目/模块/业务域/标签建立多级目录结构，提供模糊搜索与组合筛选功能；
- 3) 应支持按照预设顺序执行单个或多个脚本，并支持结果查看。

2 级：

- 1) 应支持脚本模板化与参数化；
- 2) 应支持测试脚本与测试用例的关联；
- 3) 应具备脚本版本管理能力，记录脚本的修改历史，支持脚本回滚到指定版本；
- 4) 应具备交互式调试能力，包含断点设置、变量监视、单步执行等；
- 5) 应具备脚本依赖管理能力，可以识别脚本之间的依赖关系下载依赖包；
- 6) 宜提供场景化脚本模板库，覆盖 UI 自动化测试、性能测试、接口测试等典型场景，支持模板参数预设与自定义扩展。

3 级：

- 1) 应支持脚本的自动修复（如元素定位失效后自动更新）；
- 2) 应具备跨环境执行能力、数据管理能力。脚本与数据进行分离，脚本可跨环境执行；
- 3) 应具备生成脚本执行统计报告的能力，统计脚本执行的成功率、失败率、平均执行时间等指标；
- 4) 宜将通用的脚本封装脚本，方便其它脚本调用，提高脚本的复用性和可维护性；

- 5) 应具备脚本与持续集成/持续交付（CI/CD）工具集成能力，实现自动化测试脚本与项目构建、部署流程的无缝对接；
- 6) 可支持 AI 驱动脚本优化（减少冗余步骤）。

9.1.3. 测试数据自动化

1 级：

- 1) 应支持测试数据的增删改查；
- 2) 应支持测试数据备份与恢复；
- 3) 应支持按数据类型（如数值型、字符型、日期型等）/所属业务领域（如金融业务数据、电商订单数据、用户信息数据等）等对测试数据进行分类管理；
- 4) 应支持通过简单的规则设定（如数值范围、字符长度等）自动生成基础的测试数据；
- 5) 应支持测试数据基础属性管理，包括数据来源、格式、数据种类等基础信息录入与展示；
- 6) 应支持测试数据检索功能，支持按关键字段进行简单查询；
- 7) 宜支持基础数据脱敏（如敏感信息替换）。

2 级：

- 1) 应支持测试数据敏感性分级管理，建立敏感数据脱敏机制；
- 2) 应实现测试数据自动生成工具集成，支持参数化数据构造；
- 3) 应建立测试数据质量评估体系，支持数据有效性校验规则配置；
- 4) 应支持对测试数据的版本控制与变更历史记录；
- 5) 应支持按测试用例与数据的关联关系进行管理与展示；
- 6) 应实现测试数据与测试用例的关联映射管理，支持多对多关系配置。

3 级：

- 1) 应实现测试数据全生命周期管理，包括创建、使用、归档、销毁流程；
- 2) 应建立测试数据知识图谱，展示数据间关联关系与业务影响；
- 3) 应支持多环境测试数据同步管理，确保不同测试环境数据一致性；
- 4) 应支持数据合规性检查（如 GDPR、隐私保护）；
- 5) 宜支持智能测试数据推荐，基于 AI 算法预测高频使用数据；
- 6) 宜支持基于人工智能的测试数据智能分析与异常检测；
- 7) 可支持与生产数据的智能同步（脱敏后复用）。

9.2. 流程自动化（新增）

9.2.1. 任务管理

1 级：

- 1) 应支持创建测试任务，宜提供任务优先级设置功能，并根据不同测试类型建立相应测试任务；
- 2) 应实现测试任务状态跟踪，支持待处理/执行中/已完成等状态标识；
- 3) 应支持按测试任务优先级进行管理和展示；
- 4) 应支持测试任务的截止时间设置与提醒；
- 5) 宜提供任务分配机制，支持测试类型分配测试资源。

2 级：

- 1) 应支持测试任务依赖关系管理，可配置前置任务及执行顺序；
- 2) 应支持分布式批量执行并保存执行记录；
- 3) 应支持执行策略配置，包括串行、并行，定时执行等；
- 4) 应建立测试任务执行规范库，提供标准执行流程模板；
- 5) 应支持测试任务的进度记录与展示、暂停、停止、重试等；
- 6) 应支持测试任务的计算测试用例通过率；
- 7) 应支持测试任务与相关测试用例的关联管理与展示；
- 8) 宜支持与 CI/CD 工具（如 Jenkins、GitLab CI）集成，实现定时/触发式自动化执行

3 级：

- 1) 应支持调试执行，可设置断点、单步调试、指定起点执行等；
- 2) 应支持智能任务推荐，基于历史数据预测任务优先级；
- 3) 应建立任务执行质量评估模型，自动生成任务执行报告；
- 4) 应支持集成 DevOps 全流程，实现资源智能调度（如云环境弹性扩展）与流水线的自动触发；
- 5) 宜支持多任务资源优化调度，基于负载预测分配执行资源；
- 6) 应支持测试任务与缺陷管理系统的深度集成与联动；
- 7) 应支持基于历史数据的测试任务规划与预测。

9.2.2. 过程监控

1 级：

- 1) 应支持测试执行过程实时监控，展示关键节点状态；

- 2) 应提供基础执行指标监控（如通过率/失败率/执行速度）；
- 3) 应支持异常事件即时报警（如用例失败/服务中断）；
- 4) 应实现测试环境基础状态监控（CPU/内存/网络）。

2 级：

- 1) 应提供测试过程审计追踪，记录关键操作日志；
- 2) 应支持自定义监控规则配置，可设置多级预警阈值；
- 3) 应实现测试资源利用率监控，分析执行效率瓶颈；
- 4) 应支持失败重试配置，可以根据配置进行失败重试；
- 5) 应支持多维度质量指标监控（缺陷密度/覆盖率/波动率）。

3 级：

- 1) 应支持与其他系统集成的监控与联动；
- 2) 应提供根因分析辅助，自动关联异常相关要素；
- 3) 应支持监控数据可视化大屏，支持多屏联动分析；
- 4) 宜支持智能异常检测，基于 AI 算法预测潜在风险；
- 5) 宜实现测试过程优化建议，基于监控数据生成改进方案。

9.3. 环境自动化

9.4. 结果自动化（新增）

1 级：

- 1) 应具备自动收集测试过程数据和结果，宜可视化展示测试过程，自动统计分析测试结果；
- 2) 应支持生成基础测试报告，包含开始时间、结束时间、测试用例执行结果、被测系统信息、缺陷数量等；
- 3) 报告应包含关键测试指标，如用例覆盖率、测试通过率、失败用例数、执行耗时、缺陷分布、环境配置信息等；
- 4) 应支持报告导出，可提供多格式导出，如 WPS、PDF、XLSX、HTML；
- 5) 宜支持执行结果明细展示，如测试环境、执行状态、失败原因、是否已提交缺陷等。

2 级：

- 1) 应支持执行结果的统计与筛选，包括执行时间、通过率等；
- 2) 应支持测试报告权限控制，确保敏感内容仅授权角色可见；

- 3) 应支持测试报告分级生成，如单次任务报告、迭代周期功能模块报告、项目整体报告；
- 4) 应支持报告的统一命名和归档，方便监管和溯源；
- 5) 应提供测试报告模板自定义配置功能，支持按需配置展示报告内容，如分析维度、可视化图表类型；
- 6) 应支持多维度分析，如缺陷等级分布、模块缺陷密度、测试覆盖率统计；
- 7) 宜具备历史报告对比分析能力，能够研判趋势变化，如识别重复缺陷模式；
- 8) 宜支持测试结果与测试需求的关联映射，确保需求实现完整性验证和质量溯源。

3 级：

- 1) 应支持测试报告实时生成，并与流水线测试执行进展同步；
- 2) 应支持测试结果与缺陷管理系统（如 JIRA、Bugzilla、Tapd、禅道等）联动，能够自动触发修复工单实行跟踪闭环；
- 3) 应支持测试报告可视化图表展示，如折线图、拓扑图，确保性能指标直观呈现；
- 4) 应支持报告内容交互式动态钻取，能够从摘要层逐级下钻至代码段落；
- 5) 宜支持智能化生成改进建议，如高危模块测试增强、缺陷修复优先级建议；
- 6) 宜支持跨项目/跨团队的测试报告聚合分析，生成组织级质量效能全景视图；
- 7) 宜支持智能告警报告实时推送，针对关键指标异常自动触发专项分析；
- 8) 宜支持与需求管理/DevOps 平台无缝对接，实现报告结论自动生成优化工单；
- 9) 宜提供自然语言查询接口，如通过 Chatbot 问答形式获取定制化报告片段。

9.5. 风险管理

9.5.1. 缺陷管理

1 级：

- 1) 应支持自动化+人工记录缺陷基础属性，包括缺陷标题、描述、严重等级、影响范围等；
- 2) 应实现缺陷全生命周期管理流程，如缺陷创建、分配、修复、验证、关闭等基本状态流转，支持手动更新状态变更记录；
- 3) 应支持缺陷标签化管理及分类分级，如功能缺陷、性能缺陷、安全漏洞；
- 4) 应支持缺陷基础统计视图，展示缺陷分布及清单明细，如按状态、按模块、按责任人等分布；
- 5) 应提供缺陷数据权限控制，确保敏感缺陷信息访问权限隔离。

2 级：

- 1) 应支持缺陷与测试用例、产品需求、代码提交记录等事件的关联映射;
- 2) 应具备专业缺陷管理工具,支持依据不同缺陷类型设置差异化处理流程,满足自动化状态流转;
- 3) 应支持缺陷原因分析能力,记录原因类型,如设计缺陷、代码逻辑错误、环境配置问题;
- 4) 应支持缺陷趋势分析;
- 5) 应具备跨项目缺陷数据聚合能力,生成组织级缺陷质量基线报告;
- 6) 应支持缺陷与 CI/CD 流水线联动,自动化创建缺陷工单。

3 级:

- 1) 应具备缺陷自动化分类能力,自动识别缺陷类型、预测缺陷潜在影响范围;
- 2) 应提供缺陷修复自动化建议,如依据历史相似缺陷及知识库,自动生成并推荐修复建议方案;
- 3) 应支持缺陷修复优先级动态调整,如结合业务影响度、修复成本的多维度评估;
- 4) 宜支持缺陷自愈能力,如环境配置类缺陷实现自动修复,并对修复效果进行量化评估。

9.5.2. 失败根因分析

9.6. 测试效果

1 级:

——应支持图表、表格等多种样式展示统计数据。

2 级:

- 1) 应支持建设情况度量,包括脚本建设数量、各类型脚本占比、脚本稳定性等统计;
- 2) 应支持执行情况度量,包括执行成功率、失败情况占比情况等。

3 级:

- 1) 宜支持资产健康度度量,包括脚本变更频率、脚本复用率、脚本评审通过率、用例替代率、缺陷发现等;
- 1) 可支持按照项目、按照人员等多角度度量,如投入产出比、人力替代、人力节省等;

10. DevOps 一体化协同

10.1. 敏捷管理

10.2. API 管理

10.3. 评审管理

10.4. 版本管理

10.5. 代码管理

10.5.1. 代码统一管理与版本控制

1 级：

- 1) 应使用集中式代码仓库（如 GitLab、GitHub、Gitee）管理所有测试代码和生产代码；
- 2) 应禁止直接向主分支（如 main/master）提交代码，必须通过合并请求（Merge Request/Pull Request）；
- 3) 应为每个代码仓库配置基础保护规则（如分支保护、强制代码审查）；
- 4) 应支持代码版本与测试版本的基线对齐（如 Git Tag 标记测试验证通过的版本）；
- 5) 应提供清晰的版本命名规范（如 v1.0.0、release-20240501）；
- 6) 宜通过提交信息（Commit Message）关联需求或测试任务（如 [T-123] Fix login bug）。

2 级：

- 1) 应实现多仓库协同管理（如微服务架构下的子模块依赖）；
- 2) 应实现代码版本与测试环境、制品库、文档的自动化映射（如通过 Helm Chart 关联 Docker 镜像、测试报告和 API 文档版本）；
- 3) 可支持自动化基线对齐（如代码合并后自动更新依赖版本）。

3 级：

——应实现智能代码变更分析，如自动分析 git diff 内容，识别变更类型（功能/缺陷/配置），推荐对应测试策略等。

10.5.2. 代码质量管理

1 级：

- 1) 应基于国家或行业标准建立企业自身的代码规范；
- 2) 应在 DevOps 平台中配置静态代码扫描工具进行质量检查，且配置的静态代码扫描工具至少可以确认检出问题所在的文件位置、具体的代码行数并给出相应的修复建议辅助开发人员进行问题修复；
- 3) 应设置质量门禁，阻止严重问题代码进入主干或告警、阻断对应流水线；
- 4) 应在 DevOps 平台中记录检出问题的流转情况并定期跟踪处理进度；
- 5) 应在 DevOps 平台中建立代码评审机制，所有变更需经过评审；
- 6) 应支持代码质量的实时可视化（包含开发、测试、生产全阶段数据）；
- 7) 宜对核心模块代码进行圈复杂度监控。

2 级：

- 1) 应实现模块化质量评估（按微服务/组件分级管理质量指标）；
- 2) 应在 DevOps 平台中建立代码健康度趋势分析（至少包含 6 个月历史数据），基于检出的质量问题做好数据运营；
- 3) 应根据检出质量问题做好人员培训，至少一月一次，逐步养成安全开发的意识。

3 级：

- 1) 应在 DevOps 平台中配置智能化的静态代码扫描工具，AI 辅助静态代码扫描工具完成误报判定并生成修复代码，减少检出问题的闭环时间，提高闭环效率，避免检出问题堆积；
- 2) 宜实施自适应质量门禁，DevOps 平台内嵌 AI 能力，根据代码分支对应的项目，识别项目在企业内部的分类分级情况，自适应配置质量门禁；
- 3) 宜建立质量知识图谱（关联历史缺陷、最佳实践、优化方案），方便研发人员持续学习，提升安全开发意识。

10.5.3. 代码安全管理

1 级：

- 1) 应建立代码安全开发规范（如输入验证、密码存储、错误处理等基本安全要求）；
- 2) 应在 DevOps 平台中配置威胁建模工具（STAC），根据企业自身情况进行威胁建模分析，生成基础安全需求文档、安全测试用例；

- 3) 应在 DevOps 平台中配置静态代码扫描工具（SAST）自动化进行代码安全扫描，并周期性进行人工代码审计；
- 4) 应建立第三方组件安全评估机制，如在 DevOps 平台中配置软件成分分析工具（SCA）自动化对代码中引用的第三方组件进行自动化安全扫描；
- 5) 应在 DevOps 平台中配置交互式应用安全检测工具（IAST）在自动化测试过程中对运行时的应用程序进行实时安全监控，定位出现安全问题的具体代码模块；
- 6) 应禁止在代码中硬编码敏感信息（如密码、API 密钥、加密密钥）；
- 7) 应实施基础访问控制（如代码仓库的读写权限分离）；
- 8) 应在 DevOps 平台中记录并跟踪处理安全扫描发现的高危漏洞，对漏洞的全生命周期进行可视化运营；
- 9) 应定期对相关人员进行安全开发意识培训（包括漏洞成因、漏洞危害、漏洞复现、代码外泄危害等）；
- 10) 宜对生产环境配置进行安全审查（如数据库连接字符串、服务端点）。

2 级：

- 1) 应建立、健全安全开发体系管理制度，多维度对代码安全进行管理；
- 2) 应实施软件物料清单（SBOM）管理所有组件依赖关系；
- 3) 宜实施基础设施即代码（IaC）的安全扫描；
- 4) 应确保 DevOps 平台中包含漏洞全生命周期的数据或能直接与相应平台对接。

3 级：

- 1) 应在 DevOps 平台中配置智能化威胁建模工具（STAC），根据企业自身情况进行威胁建模分析，自动生成基础安全需求文档、安全测试用例；
- 2) 应在 DevOps 平台中配置智能化静态代码扫描工具（SAST）自动化进行代码安全扫描，在支持自动化误报判断和修复建议生成的基础上，可以将周期性进行人工代码审计的结果与工具扫描结果进行自动化除重、重点漏洞告警等；
- 3) 应引入智能化的软件成分分析工具（SCA），可以通过自然语言的方式快速检索关注信息；
- 4) 应在 DevOps 平台中配置智能化交互式应用安全检测工具（IAST），在自动化测试过程中可以自动与威胁建模工具（STAC）生成的生成基础安全需求文档、安全测试用例结合；
- 5) 宜在 DevOps 平台中加入 AI 能力，在所有工具的数据进行汇总并持续运营的基础上，通过 AI 的能力为团队赋能，使每个人都具备软件代码安全专家的能力，降低工具的使用门槛和跨部门协同的频率，高效地完成漏洞闭环。

10.6. 制品管理

1 级：

- 1) 应在 DevOps 平台建立统一制品仓库（如 Nexus、Harbor）存储所有构建产物并与其他各个阶段无缝衔接；
- 2) 应实现制品版本与代码版本严格绑定（如 app-1.0.0.jar 对应 git tag v1.0.0）；
- 3) 应配置镜像扫描、软件成分分析工具（SCA）对制品库中的镜像、组件包进行管理；
- 4) 应记录制品构建环境信息（如 JDK 版本、构建时间）。

2 级：

- 1) 应实现多环境制品晋升流程（如 DEV→TEST→PROD）并配置多个制品库，支持制品库的无缝摆渡；
- 2) 应自动生成制品 SBOM（软件物料清单）；
- 3) 应实现制品扫描与流水线卡点联动；
- 4) 宜建立制品依赖关系图谱（如 Docker 镜像层分析）。

3 级：

- 1) 应具备自适应漏洞修复能力，当扫描到制品漏洞时，AI 自动生成修复方案（如依赖升级建议）；
- 2) 可建设智能晋升决策，根据测试结果和业务指标（如 SLA 达标率），AI 自动判断是否晋升到生产环境。

10.7. 安全管理

1 级：

- 1) 应实施基础访问控制，并对所有账户权限进行颗粒度划分；
- 2) 应记录关键操作审计日志（保留至少 180 天）；
- 3) 应根据企业内部软件分类分级情况建设流水线安全门禁；
- 4) 应定期对 DevOps 平台进行工具与人工的渗透测试；
- 5) 应对使用者进行安全意识培训，避免用户名密码外泄到互联网等情况出现；
- 6) 应在流水线中的各个阶段嵌入相应的安全工具（包括不限于静态代码扫描工具（SAST）、软件成分分析工具（SCA）、交互式应用安全检测工具（IAST）、镜像扫描工具等）。

2 级：

- 1) 应实施零信任架构（如 SPIFFE 身份认证）；

- 2) 应在网络层做好网络隔离，避免因工作节点失陷导致整个平台被控制；
- 3) 应定期执行安全基线检查（如 CIS Benchmark）；
- 4) 宜支持多因素认证（MFA），关键操作需二次验证；
- 5) 宜采用欺骗防御技术（如蜜罐诱捕攻击者），提高系统安全性，增加应急响应时间。

3 级：

- 1) 应实施 RASP（运行时应用自保护），拦截内存马、注入攻击；
- 2) 应建设容器运行时安全监测，实时监控容器的异常行为；
- 3) 应支持攻击链自动阻断（如联动防火墙封禁恶意 IP）。

10.8. 流水线管理

1 级：

- 1) 应设置超时中断机制（默认不超过 60 分钟）；
- 2) 应提供流水线执行可视化看板；
- 3) 应保留最近 30 天流水线日志；
- 4) 应实现代码提交触发的基础构建流水线（如 Git Hook + Jenkins）；
- 5) 应支持多环境部署（如 DEV/TEST/PROD 等）；
- 6) 应记录部署版本与代码版本的对应关系；
- 7) 应支持 k8s 集群作为流水线执行环境，同时启用 RBAC 并关闭匿名访问。

2 级：

- 1) 应建立流水线性能优化机制（如缓存策略）；
- 2) 应实现流水线异常自愈（自动重试/回滚）；
- 3) 应实现多阶段审批流程（如生产发布需运维确认并通过安全审核）；
- 4) 应强制所有部署包含资源限制（requests/limits）；
- 5) 应支持资源弹性伸缩（根据负载自动调整执行节点）；
- 6) 宜采用声明式流水线定义（如 Tekton、GitLab CI）；
- 7) 应提供流水线性能分析报告（如各阶段耗时统计、性能消耗统计），在测试过程中进行压力测试。

3 级：

- 1) 应提供跨团队协同能力，实现微服务流水线编排（如服务依赖自动触发），支持混合云流水线调度（跨公有云/私有云资源调配）；
- 2) 宜建立跨流水线协同恢复机制（如微服务流水线间的依赖故障处理）；

- 3) 宜配备 AI 能力，支持智能路径优化（根据变更内容跳过无关阶段，如仅进行增量的代码检测）。
-